Documentación-errores-y-seguimiento-de-defectos

Sharon Michelle Barraza Meriño

Servicio nacional de aprendizaje (SENA)

Fontibón

Pruebas de software

Daniel Lopez

13/08/2026

TALLER 5: Documentación de errores y seguimiento de defectos

1. Objetivo

Aplicar los procesos de documentación, clasificación y seguimiento de defectos en software mediante el estándar IEEE 829 y la corrección de errores en una aplicación interactiva.

2. ACTIVIDADES

Un ***reporte de errores*** (***bug report***) no es solo un reclamo; es un documento técnico vital que le permite al equipo de desarrollo entender, reproducir y solucionar una falla. El objetivo de este taller es afianzar sus habilidades en la gestión de calidad de software mediante la aplicación práctica del estándar IEEE 829, la clasificación de severidad/prioridad, el seguimiento del ciclo de vida del defecto y la construcción de un entorno interactivo.

Plantilla Base: Formato IEEE 829

Utilice la siguiente estructura para completar las actividades que soliciten el diligenciamiento del **Reporte de Incidente de Prueba**:

Formato de Reporte de Incidente de Prueba (IEEE 829)

- **Identificador del Reporte:** *(Ejemplo: IR-LOGIN-001)*
- **Resumen del Incidente:**
    - **Tipo de Incidente:** *(Ejemplo: Fallo de validación, error visual)*
    - **Entorno de Prueba:** *(Ejemplo: Chrome v120 en Windows 11)*
    - **Versión del Software:** *(Ejemplo: v1.0.2)*
    - **Severidad:** *(Crítico, Mayor, Menor)*
    - **Prioridad:** *(Alta, Media, Baja)*
- **Descripción del Incidente:**
    - **Paso del Procedimiento de Prueba:** *(Paso exacto que generó el fallo)*
    - **Entradas (Inputs):** *(Datos ingresados)*
    - **Resultados Esperados:** *(Lo que la aplicación debería haber hecho)*
    - **Resultados Actuales:** *(Lo que la aplicación hizo de forma incorrecta)*

- **Estado:** *(Nuevo, Asignado, En Progreso, Resuelto, En Prueba, Cerrado)*

- **Impacto:** *(Cómo afecta el defecto a las pruebas o al negocio)*
- **Recomendaciones:** *(Acción sugerida para solucionar el problema)*

Actividad 1: Creación de un Reporte de Incidente (Estándar IEEE 829)

Escenario de prueba:

Imagine que está realizando pruebas sobre un formulario de registro de usuario. Al ingresar una contraseña de menos de 8 caracteres, nota que el sistema no muestra un mensaje de error y permite continuar con el registro.

Instrucción:

Diligencie completamente la Plantilla Base IEEE 829 descrita anteriormente con la información correspondiente a esta falla. En la pestaña “Control de pruebas (IEEE 829)” hay un espacio listo para diligenciar esta tarea antes de las tareas relacionadas y diligenciadas como ejemplo.

- La actividad 1 la resolví en la plantilla

Actividad 2: Clasificación de Severidad y Prioridad

Instrucción:

Para cada uno de los siguientes 3 escenarios:

1. Asigne un nivel de Severidad (*Crítico, Mayor, Menor*).
2. Asigne un nivel de Prioridad (*Alta, Media, Baja*).
3. Redacte al menos un párrafo de justificación para su elección.

- **Escenario A:** La página de inicio de sesión no carga y muestra un mensaje de error HTTP 500.

- **Nivel de Severidad**: Crítico
- **Nivel de Prioridad:** Alta
- **Justificación:** Si la página de inicio de sesión no carga y muestra error es imposible que el cliente pueda entrar al sistema y no pueda realizar su proceso, tiene dificultades tanto el cliente como la persona que vende ese sistema y es muy crítico ya que de las dos partes por decirlo así no están ganando y alta prioridad ya que es un problema que se tiene que solucionar para que no se ven afectadas esa dos partes

- **Escenario B:** El botón *"Añadir al carrito"* no funciona al ingresar desde Microsoft Edge, pero opera correctamente en Chrome y Firefox.

- **Nivel de Severidad**: Mayor
- **Nivel de Prioridad:** Media
- **Justificación:** Es un problema para el cliente si quiere añadir al carrito pero que no le funcione en Microsoft Edge si es el único navegador web que utilice o tenga a su disposición pero si funciona correctamente en Chrome y Firefox es porque se puso el criterio de que solo sirva en esos dos navegadores, no es un problema crítico porque igual funciona en otros pero si es algo que se puede solucionar y media porque aun asi sirve ese botón

- **Escenario C:** En la página de perfil del usuario, el nombre de usuario tiene una falla ortográfica.

- **Nivel de Severidad**: Menor
- **Nivel de Prioridad:** Baja
- **Justificación:** No es un problema severo ya que es error ortográfico en el nombre del usuario que el mismo dígito a la hora de registrarse y es un pequeño problema que se puede solucionar fácilmente por eso es menor el nivel de severidad y baja la prioridad ya que esto no le afecta la página si tiene que realizar otro proceso completamente diferente

Diligencie esta solución en un documento de Word.

Actividad 3: Flujo y Ciclo de Vida del Defecto

Instrucción:

Dados los siguientes casos:

- **Caso 1: El Carrito de Compras Desaparecido**

Situación: Al agregar un producto al carrito en la app móvil, cerrar la app y volver a abrirla, el carrito aparece totalmente vacío. (*Nivel inicial: Crítico*).

- **Caso 2: El Formulario de Contacto que No Envía Correos**

Situación: El formulario de contacto de la web muestra un mensaje de éxito, pero el correo nunca llega al equipo de soporte. (*Nivel inicial: Mayor*).

- **Caso 3: Falla de Ortografía en el Encabezado Principal**

Situación: El título de la página principal dice "**Bienvenidos a la nueba tienda en línea**". (*Nivel inicial: Menor*).

Para los anteriores casos:

1. Complete la **Plantilla Base IEEE 829** con la información planteada.
2. Explique paso a paso cómo se desplaza el error a través de las 6 etapas del ciclo de vida (Nuevo ➔ Asignado ➔ En Progreso ➔ Resuelto ➔ En Prueba ➔ Cerrado).

3. Identifique qué rol del equipo (Tester/QA, Desarrollador, Líder de Proyecto/Gerente) es el responsable de realizar y autorizar cada cambio de estado.

Deberá entregar un documento de Excel (***Plantilla Base IEEE 829***) con la información completa por cada caso.

Esta adjuntado la plantilla

ACTIVIDAD 4: ANÁLISIS DE HERRAMIENTAS DE SEGUIMIENTO

INSTRUCCIÓN:

Analice el contexto de un equipo de desarrollo conformado por 10 integrantes:

1. **Cuadro Comparativo de Ventajas:** Genere y argumente **mínimo 10 ventajas** de usar una herramienta especializada (***Jira, Trello, Asana***) frente a una hoja de cálculo compartida (***Excel, Google Sheets***).

Ventaja	**Herramienta especializada (*Jira, Trello, Asana*)**	**Hoja de cálculo compartida (*Excel, Google Sheets*)**
Menor margen de error humano	Los formularios guiados y la validación de campos evitan datos incompletos, cálculos rotos o registros duplicados	Expuesta a fórmulas dañadas, datos incompletos o registros duplicados por error de digitación
Filtros combinados	Permite cruzar criterios como severidad, estado, módulo y responsable en una sola búsqueda	Obliga a revisar columna por columna de forma manual, lo cual se vuelve lento con muchos registros
Crecimiento sin fricción	Sigue funcionando con fluidez aunque el equipo pase de 10 a 30 o 50 integrantes y se sumen nuevos proyectos	Se vuelve difícil de manejar y de mantener ordenada a medida que crecen las filas y los usuarios
Reportes automáticos	Los tableros y gráficos de avance se actualizan solos a medida que cambian los tickets	Obliga a reconstruir tablas y gráficos dinámicos manualmente cada vez que se necesitan
Permisos diferenciados	Cada usuario puede tener un nivel de acceso distinto (ver, comentar, editar, cerrar) según su rol	El control de acceso es básico (protección de hoja o archivo) y difícil de segmentar por rol
Conexión con el stack de desarrollo	Se integra de forma directa con repositorios como GitHub o GitLab y con los	No tiene integración nativa; cualquier conexión exige macros o pasos manuales

	canales de comunicación del equipo	
Vistas adaptadas a cada persona	El mismo tablero puede mostrarse como lista, kanban o vista de sprint según lo que necesite cada integrante	Ofrece una sola vista de filas y columnas; hay que duplicar o filtrar manualmente para cada rol
Flujos que se disparan solos	Cuando un ticket cambia de estado, notifica automáticamente a los responsables	No genera notificaciones automáticas; el aviso hay que darlo manualmente por correo o chat
Registro histórico confiable	Cada modificación, comentario o reasignación queda guardada automáticamente con fecha y autor	Depende de que alguien anote manualmente quién cambió qué; es fácil de omitir o perder
Trabajo simultáneo sin choques	Varias personas pueden abrir y modificar el mismo ticket a la vez sin sobrescribir el trabajo de nadie	Existe alto riesgo de sobrescritura de datos y bloqueo del archivo cuando varios editan a la vez

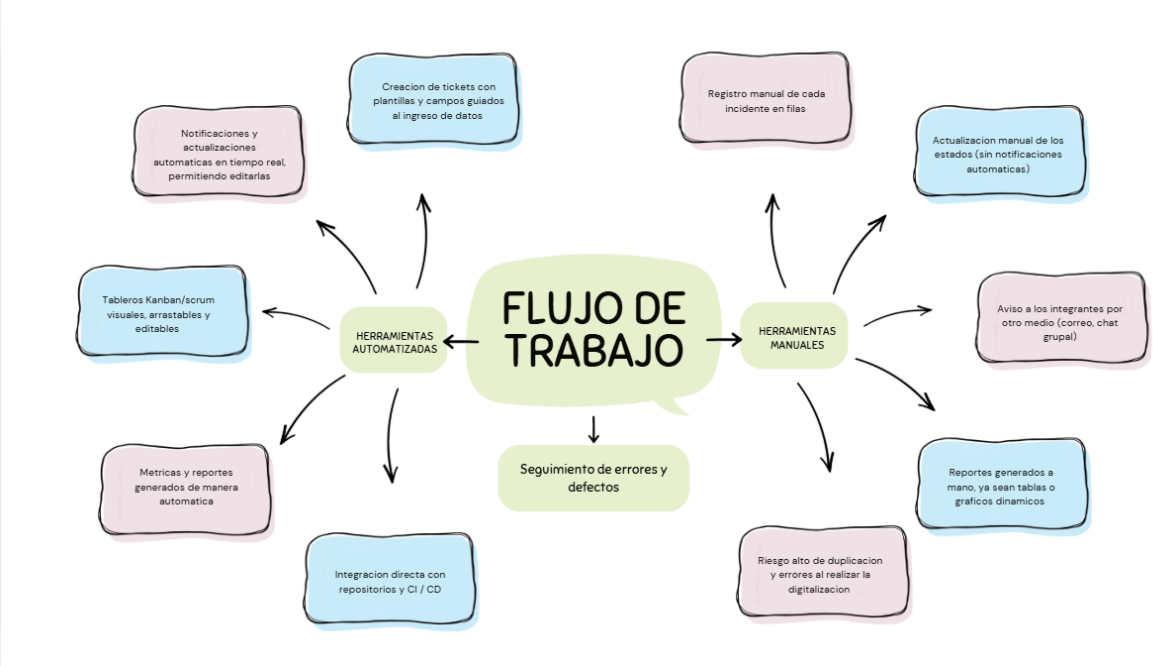
2. **Análisis de Riesgos/Desventajas:** Identifique y explique **mínimo 5 desventajas o retos** al implementar una herramienta especializada de seguimiento.

Desventaja	**Descripcion**
Tiempo de adaptación	El equipo necesita entrenarse en la lógica de la herramienta (estados, campos, automatizaciones), lo que retrasa un poco la productividad al inicio
Inversión económica	El pago de licencia por cada uno de los 10 integrantes puede sumar un costo mensual considerable frente a una hoja de cálculo, que es gratuita
Cambio de hábitos	Algunas personas del equipo, acostumbradas a Excel durante años, pueden tardar en confiar en la nueva forma de trabajar
Necesidad de conexión estable	Al ser una herramienta en la nube, cualquier falla de internet o del proveedor deja al equipo sin acceso al seguimiento de errores por un tiempo

Puesta a punto inicial	Configurar campos personalizados, permisos y flujos de trabajo toma tiempo antes de empezar a usar la herramienta de forma óptima

3. **Mapa Mental:** Elabore un mapa mental donde compare el flujo de trabajo de las **Herramientas Automatizadas/Especializadas** versus las **Herramientas Manuales/Semiautomáticas**.

Deberá entregar un documento de Word con el desarrollo de esta actividad.



Actividad 5: Construcción de Entorno de Pruebas React y Documentación

Paso 1: Crear el proyecto

Escriba en la terminal los siguientes comandos:

```
npx create-react-app control_pruebas
cd control_pruebas
npm install
```

Paso 2: Estructura del proyecto

```
src/
 ├── App.js
 ├── components/
 │    ├── Navbar.js
 │    ├── ProductList.js
 │    ├── Cart.js
 │    └── Footer.js
```

Paso 3: Configurar los documentos de la estructura:

components/Navbar.js

```jsx
import React from "react";

function Navbar() {
  return (
    <nav className="navbar">
      {/* ❌ Error 1: imagen rota */}
      <img src="/logo-tienda.png" alt="logo" width="80" />

      {/* ❌ Error 2: texto muy pegado */}
      <ul>
        <li><a href="#home">Inicio</a></li>
        <li><a href="#productos">Productos</a></li>
        <li><a href="#contacto">Contacto</a></li>
      </ul>
    </nav>
  );
}

export default Navbar;
```

components/ProductList.js

```jsx
import React from "react";

function ProductList() {
  return (
    <div className="productos">
      <h2>Lista de Productos</h2>

      {/* ❌ Error 3: imagen rota */}
      <img src="/camisa.png" alt="Camisa" width="150" />

      {/* ❌ Error 4: texto con mismo color que el fondo */}
      <p className="texto-invisible">Precio: $50.000</p>
```

```
      {/* ❌ Error 5: elementos desordenados visualmente */}
      <ul>
        <li>Pantalón - $70.000</li>
        <li>Camiseta - $30.000</li>
        <li>Zapatos - $120.000</li>
        <li>Bolso - $90.000</li>
      </ul>

      {/* ❌ Error 6: botón muy pequeño */}
      <button className="boton-comprar">Comprar</button>
    </div>
  );
}

export default ProductList;
```

components/Cart.js

```
import React from "react";

function Cart() {
  return (
    <div className="cart">
      <h2>Carrito de Compras</h2>

      {/* ❌ Error visual 11: tabla descuadrada */}
      <table className="tabla-carrito">
        <tr>
          <th>Producto</th>
          <th>Cantidad</th>
          <th>Precio</th>
        </tr>
        <tr>
          <td>Camiseta</td>
          <td>2</td>
          <td>$60.000</td>
        </tr>
        <tr>
          <td>Zapatos</td>
          <td>1</td>
          <td>$120.000</td>
        </tr>
        {/* ❌ Error visual 12: fila con datos incompletos */}
        <tr>
          <td>Bolso</td>
          <td></td>
          <td></td>
        </tr>
      </table>
```

```
      {/* ❌ Error visual 13: botón enorme que rompe diseño */}
      <button className="btn-pagar">Pagar</button>

      {/* ❌ Error visual 14: texto que se sale del contenedor */}
      <p className="texto-largo">
        Este es un mensaje de prueba que no tiene saltos de línea ni está
        limitado en ancho, por lo que va a seguir extendiéndose y dañando la
        maquetación de la página hasta que ocupe todo el espacio horizontal.
      </p>
    </div>
  );
}

export default Cart;
```

components/Footer.js

```
import React from "react";

function Footer() {
  return (
    <footer className="footer">
      {/* ❌ Error 7: texto cortado */}
      <p>© 2025 Tienda de Ropa - Todos los dere</p>

      {/* ❌ Error 8: enlace sin destino */}
      <a>Política de Privacidad</a>

      {/* ❌ Error 9: icono mal alineado */}
      <div className="redes">
        <span>📷 Instagram</span>
        <span>📘 Facebook</span>
        <span>🐦 Twitter</span>
      </div>

      {/* ❌ Error 10: fondo del footer demasiado oscuro con texto oscuro
*/}
      <p className="texto-oscuro">Contacto: info@tienda.com</p>
    </footer>
  );
}

export default Footer;
```

App.js

```
import React from "react";
import Navbar from "./components/Navbar";
import ProductList from "./components/ProductList";
import Footer from "./components/Footer";
import Cart from "./components/Cart"; // Nuevo componente
import "./App.css";
```

```jsx
function App() {
  return (
    <div>
      <Navbar />
      <h1 className="titulo">Bienvenido a la Tienda</h1>
      <ProductList />
      <Cart /> {/* Carrito agregado aquí */}
      <Footer />
    </div>
  );
}

export default App;
```

App.css

```css
/* Estilos del carrito de compras con errores intencionales */

.cart {
  margin: 20px;
  border: 2px solid red; /* borde demasiado fuerte */
  padding: 10px;
  width: 90vw;
}

.tabla-carrito {
  border-collapse: collapse;
  width: 120%; /* ❌ la tabla se sale del contenedor */
}

.tabla-carrito th, .tabla-carrito td {
  border: 1px solid black;
  padding: 5px;
}

.btn-pagar {
  width: 500px;  /* botón exageradamente grande */
  height: 100px;
  background: yellow;
}

.texto-largo {
  white-space: nowrap; /* ❌ hace que el texto no haga salto de línea */
}
```

Paso 4: Para ejecutar la aplicación, en la terminal escriba:

```
npm start
```

Paso 5: Registro y Solución de Defectos

1. **Documentación:** Realice la inspección visual y funcional de la aplicación web y construya un reporte individual usando la ***Plantilla IEEE 829*** por cada uno de los errores identificados (todos los errores).
2. **Corrección de Código:** Modifique los componentes ***React*** y el archivo "***App.css***" para resolver cada problema presentado.
3. **Cierre:** Actualice el estado de cada reporte en sus documentos a (***Cerrado***) adjuntando captura de pantalla como evidencia del código funcional.

   Plantilla adjuntada con todos los errores y el estado cerrado

Navbar.js

src > components > Navbar.js > ...

```
1   import React from "react";
2
3   function Navbar() {
4     return (
5       <nav className="navbar">
6         <img
7           src="images.png"
8           alt="logo"
9           width="80"
10          onError={(e) => {
11            e.target.onerror = null;
12            e.target.src = "images.png";
13          }}
14        />
15
16        <ul className="nav-links">
17          <li><a href="#home">Inicio</a></li>
18          <li><a href="#productos">Productos</a></li>
19          <li><a href="#contacto">Contacto</a></li>
20        </ul>
21      </nav>
22    );
23  }
24
25  export default Navbar;
26
```

Inicio Productos Contacto

Bienvenido a la Tienda

Lista de Productos



Precio: $50.000

- Camiseta - $30.000
- Pantalón - $70.000
- Bolso - $90.000
- Zapatos - $120.000

Comprar

Navbar.js ProductList.js

src > components > ProductList.js > ProductList

```jsx
import React from "react";

function ProductList() {
  const productos = [
    { nombre: "Camiseta", precio: 30000 },
    { nombre: "Pantalón", precio: 70000 },
    { nombre: "Bolso", precio: 90000 },
    { nombre: "Zapatos", precio: 120000 },
  ];

  return (
    <div className="productos">
      <h2>Lista de Productos</h2>

      <img
        src="camisa.png"
        alt="Camisa"
        width="150"
        onError={(e) => {
          e.target.onerror = null;
          e.target.src = "camisa.png";
        }}
      />

      <p className="texto-visible">Precio: $50.000</p>

      <ul className="lista-productos">
        {productos.map((p) => (
          <li key={p.nombre}>
            {p.nombre} - ${p.precio.toLocaleString("es-CO")}
          </li>
        ))}
      </ul>

      <button className="boton-comprar">Comprar</button>
    </div>
  );
}

export default ProductList;
```


Inicio
Productos
Contacto
Bienvenido a la Tienda
Lista de Productos
Precio: $50.000
Camiseta - $30.000
Pantalón - $70.000
Bolso - $90.000
Zapatos - $120.000
Comprar
Carrito de Compras
Producto
Cantidad
Precio

Navbar.js ProductList.js Cart.js

src > components > Cart.js > Cart

```jsx
import React from "react";

function Cart() {
  const items = [
    { producto: "Camiseta", cantidad: 2, precio: "$60.000" },
    { producto: "Zapatos", cantidad: 1, precio: "$120.000" },
    { producto: "Bolso", cantidad: 1, precio: "$90.000" },
  ];

  return (
    <div className="cart">
      <h2>Carrito de Compras</h2>

      <table className="tabla-carrito">
        <thead>
          <tr>
            <th>Producto</th>
            <th>Cantidad</th>
            <th>Precio</th>
          </tr>
        </thead>
        <tbody>
          {items.map((item) => (
            <tr key={item.producto}>
              <td>{item.producto}</td>
              <td>{item.cantidad}</td>
              <td>{item.precio}</td>
            </tr>
          ))}
        </tbody>
      </table>

      <button className="btn-pagar">Pagar</button>

      <p className="texto-largo">
        Este es un mensaje de prueba que ahora respeta el ancho del
        contenedor y hace salto de línea correctamente sin dañar la
        maquetación de la página.
      </p>
    </div>
  );
}

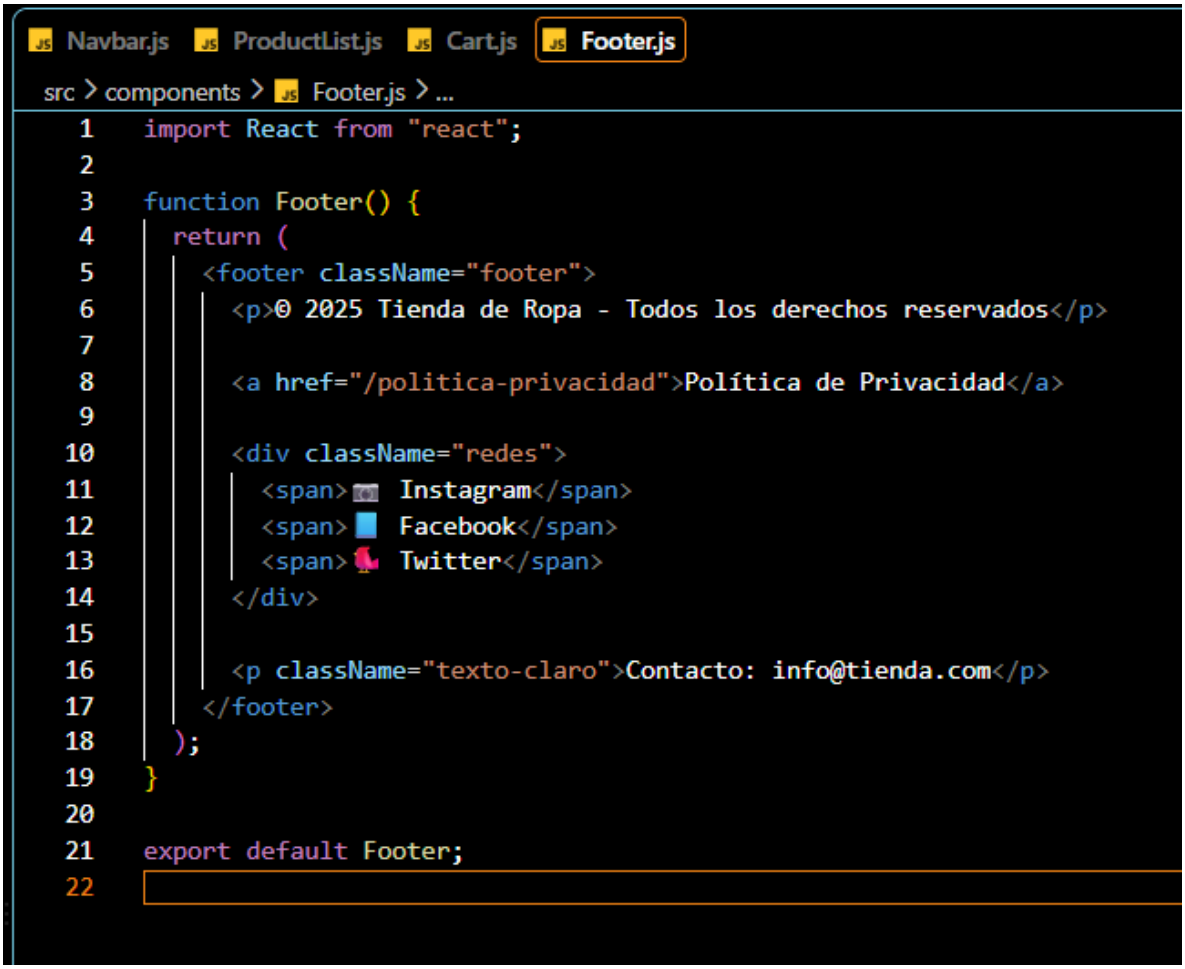
```

Carrito de Compras

Producto	Cantidad	Precio
Camiseta	2	$60.000
Zapatos	1	$120.000
Bolso	1	$90.000

Pagar

Este es un mensaje de prueba que ahora respeta el ancho del contenedor y hace salto de línea correctamente sin dañar la maquetación de la página.



© 2025 Tienda de Ropa - Todos los derechos reservados

Política de Privacidad

Instagram Facebook Twitter

Contacto: info@tienda.com



Inicio Productos Contacto

Bienvenido a la Tienda

Lista de Productos



Precio: $50.000

- Camiseta - $30.000
- Pantalón - $70.000
- Bolso - $90.000
- Zapatos - $120.000

Comprar

Carrito de Compras

Producto	Cantidad	Precio
Camiseta	2	$60.000
Zapatos	1	$120.000
Bolso	1	$90.000

Pagar

Este es un mensaje de prueba que ahora respeta el ancho del contenedor y hace salto de línea correctamente sin dañar la maquetación de la página.

© 2025 Tienda de Ropa - Todos los derechos reservados

Política de Privacidad

Instagram Facebook Twitter